

AI LAB PROGRAMS

Exam-Friendly Python & Prolog Programs

AIM • ALGORITHM • EXECUTABLE CODE • OUTPUT • EXPLANATION • MEMORY • VIVA

PROGRAM 1 — 8-Puzzle using BFS

1. AIM

To implement 8-Puzzle using BFS.

2. ALGORITHM

- Create the initial state.
- Use a queue and visited set.
- Remove the next state.
- Generate valid next states.
- Add unseen states until the goal is reached.

3. PROGRAM

```
from collections import deque

def bfs(start, goal):
    q = deque([(start, [])])
    seen = {start}
    while q:
        s, path = q.popleft()
        if s == goal:
            print(path + [s]); return
        z = s.index(0)
        for d in (-1, 1, -3, 3):
            n = z + d
            if 0 <= n < 9 and not (d in (-1,1) and z//3 != n//3):
                a = list(s); a[z],a[n] = a[n],a[z]; a = tuple(a)
                if a not in seen:
                    seen.add(a); q.append((a, path+[s]))

start = tuple(map(int, input("Enter start (0-8): ").split()))
goal = tuple(map(int, input("Enter goal (0-8): ").split()))
bfs(start, goal)
```

4. EXPECTED OUTPUT

Enter two 9-number states. Example: 1 2 3 4 0 6 7 5 8

5. CODE EXPLANATION

State = 9-tuple; 0 is blank; BFS generates legal blank moves.

6. WHAT TO REMEMBER

State + Queue + Seen + Goal.

7. VIVA QUESTIONS

What is 8-Puzzle? A sliding puzzle with 8 tiles and one blank.

Which search is used? BFS.

Why use visited/seen? To avoid repeated states.

PROGRAM 2 — 8-Queen using Backtracking

1. AIM

To implement 8-Queen using Backtracking.

2. ALGORITHM

- Choose a position/value.
- Check constraints.
- Place it and recurse.
- If it fails, undo and backtrack.
- Stop when a complete solution is found.

3. PROGRAM

```
def safe(b, r, c):
    for i in range(r):
        if b[i] == c or abs(b[i]-c) == abs(i-r):
            return False
    return True

def solve(b, r):
    if r == 8:
        print(b); return True
    for c in range(8):
        if safe(b,r,c):
            b.append(c)
            if solve(b,r+1): return True
            b.pop()
    return False

solve([],0)
```

4. EXPECTED OUTPUT

```
[0, 4, 7, 5, 2, 6, 1, 3]
```

5. CODE EXPLANATION

One queen is placed per row. safe() checks column and diagonal; append places and pop backtracks.

6. WHAT TO REMEMBER

Backtracking → Choose → Check → Recurse → Undo.

7. VIVA QUESTIONS

Which technique? Backtracking.

Why abs()? To check the diagonal condition.

What does pop() do? Removes the last queen and backtracks.

PROGRAM 3 — Water Jug using BFS

1. AIM

To implement Water Jug using BFS.

2. ALGORITHM

- Create the initial state.
- Use a queue and visited set.
- Remove the next state.
- Generate valid next states.
- Add unseen states until the goal is reached.

3. PROGRAM

```
from collections import deque

def jug(a,b,t):
    q=deque([(0,0)]); seen={(0,0)}
    while q:
        x,y=q.popleft()
        if x==t or y==t: print("Goal:",x,y); return
        s=[(a,y),(x,b),(0,y),(x,0),
            (x-min(x,b-y),y+min(x,b-y)),
            (x+min(y,a-x),y-min(y,a-x))]
        for z in s:
            if z not in seen:
                seen.add(z); q.append(z)

a=int(input("Jug A capacity: ")); b=int(input("Jug B capacity: "))
t=int(input("Target: ")); jug(a,b,t)
```

4. EXPECTED OUTPUT

Example: capacities 4 and 3, target 2 → Goal state contains 2.

5. CODE EXPLANATION

State = (x,y). Six operations: fill, empty, and transfer. BFS explores states.

6. WHAT TO REMEMBER

State + Queue + Seen + Goal.

7. VIVA QUESTIONS

What is the state? (x,y), amounts in the two jugs.

Main operations? Fill, empty, transfer.

Why min()? To respect available water and receiving capacity.

PROGRAM 4 — Crypt-Arithmetic: SEND + MORE = MONEY

1. AIM

To implement Crypt-Arithmetic: SEND + MORE = MONEY.

2. ALGORITHM

- Generate possible permutations/assignments.
- Calculate or validate each possibility.
- Keep the valid/best solution.
- Print the result.

3. PROGRAM

```
from itertools import permutations

w1,w2,res="SEND","MORE","MONEY"
letters=set(w1+w2+res)

for p in permutations(range(10),len(letters)):
    d=dict(zip(letters,p))
    if d[w1[0]]==0 or d[w2[0]]==0 or d[res[0]]==0: continue
    n1=int(''.join(str(d[x]) for x in w1))
    n2=int(''.join(str(d[x]) for x in w2))
    n3=int(''.join(str(d[x]) for x in res))
    if n1+n2==n3:
        print(d); print(n1,n2,n3); break
```

4. EXPECTED OUTPUT

```
9567 + 1085 = 10652
```

5. CODE EXPLANATION

Permutations assign unique digits to letters; leading letters cannot be zero; the arithmetic equation validates the solution.

6. WHAT TO REMEMBER

Crypt → Unique digits → Mapping → Equation.

7. VIVA QUESTIONS

Can two letters share a digit? No.

Why permutations? To try unique digit assignments.

What is the solution? SEND=9567, MORE=1085, MONEY=10652.

PROGRAM 5 — Missionaries and Cannibals using BFS

1. AIM

To implement Missionaries and Cannibals using BFS.

2. ALGORITHM

- Create the initial state.
- Use a queue and visited set.
- Remove the next state.
- Generate valid next states.
- Add unseen states until the goal is reached.

3. PROGRAM

```
from collections import deque

def safe(m,c):
    return (m==0 or m>=c) and (3-m==0 or 3-m>=3-c)

def solve():
    q=deque([(3,3,1),[]]); seen={(3,3,1)}
    moves=[(1,0),(2,0),(0,1),(0,2),(1,1)]
    while q:
        (m,c,b),path=q.popleft()
        if (m,c,b)==(0,0,0):
            print(path+[(m,c,b)]); return
        for x,y in moves:
            nm,nc=(m-x,c-y) if b else (m+x,c+y)
            if 0<=nm<=3 and 0<=nc<=3 and safe(nm,nc):
                s=(nm,nc,1-b)
                if s not in seen:
                    seen.add(s); q.append((s,path+[(m,c,b)]))

solve()
```

4. EXPECTED OUTPUT

A sequence of valid states ending at (0,0,0).

5. CODE EXPLANATION

State = (M,C,B). Five legal boat loads are tried; safe() prevents cannibals from outnumbering missionaries.

6. WHAT TO REMEMBER

State + Queue + Seen + Goal.

7. VIVA QUESTIONS

Initial state? (3,3,1).

Goal state? (0,0,0).

Boat capacity? Two people.

PROGRAM 6 — Vacuum Cleaner Problem

1. AIM

To implement Vacuum Cleaner Problem.

2. ALGORITHM

- Represent the problem state.
- Generate valid choices.
- Apply the required rule.
- Stop when the goal/output is obtained.

3. PROGRAM

```
room={"A":"D","B":"D"}
pos=input("Start room (A/B): ").upper()
while "D" in room.values():
    if room[pos]=="D":
        print("Clean",pos); room[pos]="C"
    else:
        pos="B" if pos=="A" else "A"
print("All rooms clean:",room)
```

4. EXPECTED OUTPUT

Clean A / Clean B / All rooms clean.

5. CODE EXPLANATION

Uses a simple agent: if the current room is dirty, clean it; otherwise move to the other room.

6. WHAT TO REMEMBER

Identify the state/rule first, then apply the core algorithm.

7. VIVA QUESTIONS

What is the environment? Two rooms A and B.

What does D mean? Dirty.

What does C mean? Clean.

PROGRAM 7 — Breadth First Search (BFS)

1. AIM

To implement Breadth First Search (BFS).

2. ALGORITHM

- Create the initial state.
- Use a queue and visited set.
- Remove the next state.
- Generate valid next states.
- Add unseen states until the goal is reached.

3. PROGRAM

```
from collections import deque

g={'A':['B','C'],'B':['D','E'],'C':['F'],'D':[],'E':[],'F':[]}
start=input("Start: ").upper()
goal=input("Goal: ").upper()
q=deque([start]); seen={start}
while q:
    v=q.popleft(); print(v,end=" ")
    if v==goal: break
    for n in g[v]:
        if n not in seen: seen.add(n); q.append(n)
```

4. EXPECTED OUTPUT

Example A to F: A B C D E F (stops when F is reached).

5. CODE EXPLANATION

BFS uses a FIFO queue. It visits neighbors level by level and tracks visited nodes.

6. WHAT TO REMEMBER

BFS → Queue → popleft() → visited.

7. VIVA QUESTIONS

Data structure? Queue.

FIFO means? First In, First Out.

Use? Shortest path in an unweighted graph.

PROGRAM 8 — Depth First Search (DFS)

1. AIM

To implement Depth First Search (DFS).

2. ALGORITHM

- Start from the selected node.
- Mark it visited.
- Visit one neighbor deeply.
- Backtrack when no unvisited neighbor remains.

3. PROGRAM

```
g={'A':['B','C'],'B':['D','E'],'C':['F'],'D':[],'E':[],'F':[]}  
seen=set()
```

```
def dfs(v,goal):  
    if v in seen: return False  
    seen.add(v); print(v,end=" ")  
    if v==goal: return True  
    return any(dfs(n,goal) for n in g[v])  
  
dfs(input("Start: ").upper(),input("Goal: ").upper())
```

4. EXPECTED OUTPUT

Example A to F: A B D E C F.

5. CODE EXPLANATION

DFS recursively explores one branch deeply before returning.

6. WHAT TO REMEMBER

DFS → Recursion/Stack → visited.

7. VIVA QUESTIONS

Data structure? Stack or recursion.

FIFO or LIFO? LIFO.

Difference from BFS? DFS goes deep first; BFS goes level by level.

PROGRAM 9 — Travelling Salesman Problem (TSP)

1. AIM

To implement Travelling Salesman Problem (TSP).

2. ALGORITHM

- Generate possible permutations/assignments.

- Calculate or validate each possibility.
- Keep the valid/best solution.
- Print the result.

3. PROGRAM

```
from itertools import permutations

n=int(input("Number of cities: "))
a=[list(map(int,input().split())) for _ in range(n)]
best=10**9; route=None

for p in permutations(range(1,n)):
    r=(0,)+p+(0,)
    cost=sum(a[r[i]][r[i+1]] for i in range(n))
    if cost<best: best=cost; route=r

print("Best route:",route)
print("Minimum cost:",best)
```

4. EXPECTED OUTPUT

Depends on the distance matrix.

5. CODE EXPLANATION

Fix city 0 as start, generate every route, calculate total cost, and retain the minimum.

6. WHAT TO REMEMBER

TSP → Permutations → Cost → Minimum.

7. VIVA QUESTIONS

What is TSP? Find a minimum-cost tour visiting every city once and returning to the start.

Main function? permutations().

Goal? Minimum total cost.

PROGRAM 10 — A* Search

1. AIM

To implement A* Search.

2. ALGORITHM

- Set g and heuristic h.
- Compute $f=g+h$.
- Select the smallest f.

- Expand nodes until the goal is reached.

3. PROGRAM

```
import heapq

g={'A':{'B':1,'C':3}, 'B':{'D':2}, 'C':{'D':1}, 'D':{}}
h={'A':3, 'B':2, 'C':1, 'D':0}

def astar(s,goal):
    q=[(h[s],0,s,[s])]; seen={}
    while q:
        f,c,v,path=heapq.heappop(q)
        if v==goal: print(path,c); return
        if v in seen and seen[v]<=c: continue
        seen[v]=c
        for n,w in g[v].items():
            nc=c+w; heapq.heappush(q, (nc+h[n],nc,n,path+[n]))

astar('A','D')
```

4. EXPECTED OUTPUT

Example: ['A','C','D'] 4

5. CODE EXPLANATION

A* selects the smallest $f(n)$, where $f(n)=g(n)+h(n)$. g is cost so far and h is estimated remaining cost.

6. WHAT TO REMEMBER

$A^* \rightarrow f = g + h$.

7. VIVA QUESTIONS

Formula? $f(n)=g(n)+h(n)$.

What is g ? Actual cost so far.

What is h ? Heuristic estimated cost.

PROGRAM 11 — Map Coloring using CSP

1. AIM

To implement Map Coloring using CSP.

2. ALGORITHM

- Choose a position/value.
- Check constraints.
- Place it and recurse.

- If it fails, undo and backtrack.
- Stop when a complete solution is found.

3. PROGRAM

```
colors=['R','G','B']
graph={'A':['B','C'],'B':['A','C'],'C':['A','B','D'],'D':['C']}
ans={}

def color(v):
    if v is None: print(ans); return True
    for c in colors:
        if all(ans.get(n)!=c for n in graph[v]):
            ans[v]=c
            if color(next((x for x in graph if x not in ans),None)): return True
            del ans[v]
    return False

color(next(iter(graph)))
```

4. EXPECTED OUTPUT

{'A':'R', 'B':'G', 'C':'B', 'D':'R'} (one possible solution)

5. CODE EXPLANATION

CSP uses variables (regions), domains (colors), and constraints (neighbors must differ). Backtracking assigns colors.

6. WHAT TO REMEMBER

Backtracking → Choose → Check → Recurse → Undo.

7. VIVA QUESTIONS

What is CSP? Constraint Satisfaction Problem.

Constraint? Adjacent regions cannot have the same color.

Technique? Backtracking.

PROGRAM 12 — Tic-Tac-Toe

1. AIM

To implement Tic-Tac-Toe.

2. ALGORITHM

- Represent the game state.
- Generate possible moves.
- Evaluate terminal states.

- Choose MAX or MIN according to the algorithm.

3. PROGRAM

```
b=[' ']*9
def show(): print(b[0:3]); print(b[3:6]); print(b[6:9])
def win(p):
    return any(all(b[i]==p for i in x) for x in
        [(0,1,2),(3,4,5),(6,7,8),(0,3,6),(1,4,7),(2,5,8),(0,4,8),(2,4,6)])
for turn in range(9):
    show(); p='X' if turn%2==0 else 'O'
    i=int(input(p+" position 1-9: ")) -1
    if b[i]!=' ': print("Invalid"); break
    b[i]=p
    if win(p): show(); print(p,"wins"); break
else: show(); print("Draw")
```

4. EXPECTED OUTPUT

Players enter positions 1-9; the first three-in-a-row wins.

5. CODE EXPLANATION

Board is a list of 9 cells. Each turn places X/O and win() checks the 8 winning combinations.

6. WHAT TO REMEMBER

Identify the state/rule first, then apply the core algorithm.

7. VIVA QUESTIONS

Board size? 9 cells.

Winning condition? Three same symbols in a row, column or diagonal.

Symbols? X and O.

PROGRAM 13 — Minimax

1. AIM

To implement Minimax.

2. ALGORITHM

- Represent the game state.
- Generate possible moves.
- Evaluate terminal states.
- Choose MAX or MIN according to the algorithm.

3. PROGRAM

```
def minimax(depth,maxing):
    if depth==0: return 0
    v=[minimax(depth-1,not maxing), depth if maxing else -depth]
    return max(v) if maxing else min(v)

print("Best score:",minimax(3,True))
```

4. EXPECTED OUTPUT

Best score: 2

5. CODE EXPLANATION

Minimax alternates MAX and MIN. MAX chooses the largest score; MIN chooses the smallest.

6. WHAT TO REMEMBER

Minimax $\rightarrow$ MAX = max, MIN = min.

7. VIVA QUESTIONS

MAX does what? Maximum score.

MIN does what? Minimum score.

Where used? Game-playing AI.

PROGRAM 14 — Alpha-Beta Pruning

1. AIM

To implement Alpha-Beta Pruning.

2. ALGORITHM

- Represent the game state.
- Generate possible moves.
- Evaluate terminal states.
- Choose MAX or MIN according to the algorithm.

3. PROGRAM

```
def ab(depth,alpha,beta,maxing):
    if depth==0: return 0
    if maxing:
        v=-999
        for x in [3,5]:
            v=max(v,ab(depth-1,alpha,beta,False)+x)
            alpha=max(alpha,v)
            if alpha>=beta: break
        return v
    v=999
```

```

for x in [2,4]:
    v=min(v,ab(depth-1,alpha,beta,True)-x)
    beta=min(beta,v)
    if alpha>=beta: break
return v

```

```
print(ab(3,-999,999,True))
```

4. EXPECTED OUTPUT

A numeric minimax-style score; exact value depends on the small demonstration tree.

5. CODE EXPLANATION

Alpha-beta improves minimax by pruning branches that cannot affect the final decision.

6. WHAT TO REMEMBER

Alpha-Beta $\rightarrow$ $\alpha \geq \beta \rightarrow$ prune.

7. VIVA QUESTIONS

Alpha? Best value found for MAX.

Beta? Best value found for MIN.

Pruning condition? $\alpha \geq \beta$.

PROGRAM 15 — Decision Tree Classifier

1. AIM

To implement Decision Tree Classifier.

2. ALGORITHM

- Prepare training data.
- Create the model.
- Train using fit().
- Predict using predict().

3. PROGRAM

```

from sklearn.tree import DecisionTreeClassifier

X=[[0],[1],[2],[3],[4],[5]]
y=[0,0,0,1,1,1]

m=DecisionTreeClassifier(random_state=0)
m.fit(X,y)
x=float(input("Enter value: "))
print("Predicted class:",m.predict([[x]])[0])

```

4. EXPECTED OUTPUT

For input 0-2 predicts class 0; for input 3-5 predicts class 1.

5. CODE EXPLANATION

X contains features and y contains labels. fit() trains the tree and predict() gives a class.

6. WHAT TO REMEMBER

ML $\rightarrow$ X,y $\rightarrow$ fit() $\rightarrow$ predict().

7. VIVA QUESTIONS

Library? scikit-learn.

Function to train? fit().

Function to predict? predict().

PROGRAM 16 — Feed Forward Neural Network (MLP)

1. AIM

To implement Feed Forward Neural Network (MLP).

2. ALGORITHM

- Prepare training data.
- Create the model.
- Train using fit().
- Predict using predict().

3. PROGRAM

```
from sklearn.neural_network import MLPClassifier

X=[[0,0],[0,1],[1,0],[1,1]]
y=[0,1,1,0]

m=MLPClassifier(hidden_layer_sizes=(4,),max_iter=2000,random_state=1)
m.fit(X,y)
print(m.predict([[0,1]]))
```

4. EXPECTED OUTPUT

Output is a predicted class, typically [1] for this XOR demonstration after training.

5. CODE EXPLANATION

MLPClassifier is a feed-forward neural network. X is input data, y labels, fit trains, predict classifies.

6. WHAT TO REMEMBER

ML $\rightarrow$ X,y $\rightarrow$ fit() $\rightarrow$ predict().

7. VIVA QUESTIONS

What is MLP? Multi-Layer Perceptron.

Is it supervised? Yes, when trained with labels.

Main functions? fit() and predict().

PROLOG PROGRAMS

For Prolog, save each program in a .pl file and run the query shown. These are kept short for exam execution.

PROGRAM 17 — Sum of Integers 1 to N

1. AIM

To implement Sum of Integers 1 to N using Prolog.

2. ALGORITHM

- Define the required facts/predicates.
- Define the rule or recursive relation.
- Run the query.
- Observe the result and request alternatives with `;` where applicable.

3. PROGRAM

```
sum(0,0).  
sum(N,S):- N>0, N1 is N-1, sum(N1,S1), S is N+S1.
```

```
% Query:  
% ?- sum(5,S).
```

4. EXPECTED OUTPUT

```
S = 15.
```

5. CODE EXPLANATION

Base case is sum(0,0). Recursive case reduces N by 1 and adds N.

6. WHAT TO REMEMBER

Prolog recursion → Base case + Recursive case.

7. VIVA QUESTIONS

What is recursion? A predicate calling itself until a base case.

Base case? sum(0,0).

PROGRAM 18 — Date of Birth Database

1. AIM

To implement Date of Birth Database using Prolog.

2. ALGORITHM

- Define the required facts/predicates.
- Define the rule or recursive relation.
- Run the query.
- Observe the result and request alternatives with `;` where applicable.

3. PROGRAM

```
dob(pallavi, '18-02-2006') .  
dob(ravi, '10-05-2005') .  
dob(sita, '21-08-2006') .
```

```
% Query:  
% ?- dob(pallavi,D) .
```

4. EXPECTED OUTPUT

```
D = '18-02-2006' .
```

5. CODE EXPLANATION

dob(Name,Date) stores facts. The query retrieves the date for a person.

6. WHAT TO REMEMBER

Facts + Rules + Query.

7. VIVA QUESTIONS

What is a fact? A statement stored as true.

Query? ?- dob(pallavi,D).

PROGRAM 19 — Student-Teacher-Subject

1. AIM

To implement Student-Teacher-Subject using Prolog.

2. ALGORITHM

- Define the required facts/predicates.
- Define the rule or recursive relation.
- Run the query.
- Observe the result and request alternatives with `;` where applicable.

3. PROGRAM

```
student(anu,cse) .  
student(ravi,eee) .
```

```
teacher(latha,ai).  
teaches(latha,anu,ai).
```

```
% Query:  
% ?- student(X,cse).
```

4. EXPECTED OUTPUT

```
X = anu.
```

5. CODE EXPLANATION

Facts represent students, teachers and teaching relationships.

6. WHAT TO REMEMBER

Facts + Rules + Query.

7. VIVA QUESTIONS

What is a fact? A relation known to be true.

How to retrieve CSE students? student(X,cse).

PROGRAM 20 — Planets Database

1. AIM

To implement Planets Database using Prolog.

2. ALGORITHM

- Define the required facts/predicates.
- Define the rule or recursive relation.
- Run the query.
- Observe the result and request alternatives with `;` where applicable.

3. PROGRAM

```
planet(mercury,terrestrial).  
planet(venus,terrestrial).  
planet(earth,terrestrial).  
planet(jupiter,gas_giant).
```

```
% Query:  
% ?- planet(X,terrestrial).
```

4. EXPECTED OUTPUT

```
X = mercury ; venus ; earth.
```

5. CODE EXPLANATION

Stores planet facts and queries planets by category.

6. WHAT TO REMEMBER

Facts + Rules + Query.

7. VIVA QUESTIONS

What is a predicate? A named relation such as planet/2.

How many arguments? planet has 2 arguments.

PROGRAM 21 — Towers of Hanoi

1. AIM

To implement Towers of Hanoi using Prolog.

2. ALGORITHM

- Define the required facts/predicates.
- Define the rule or recursive relation.
- Run the query.
- Observe the result and request alternatives with `;` where applicable.

3. PROGRAM

```
hanoi(1,A,B,_):- write(A),write(' -> '),write(B),nl.
hanoi(N,A,B,C):-
    N>1, N1 is N-1,
    hanoi(N1,A,C,B),
    hanoi(1,A,B,C),
    hanoi(N1,C,B,A).
```

```
% Query:
% ?- hanoi(3,left,right,middle).
```

4. EXPECTED OUTPUT

Moves all 3 disks from left to right using middle.

5. CODE EXPLANATION

Move N-1 disks to helper, move largest disk, then move N-1 disks to destination.

6. WHAT TO REMEMBER

Prolog recursion → Base case + Recursive case.

7. VIVA QUESTIONS

Technique? Recursion.

Minimum moves for n disks? $2^n - 1$.

PROGRAM 22 — Bird Can Fly / Cannot Fly

1. AIM

To implement Bird Can Fly / Cannot Fly using Prolog.

2. ALGORITHM

- Define the required facts/predicates.
- Define the rule or recursive relation.
- Run the query.
- Observe the result and request alternatives with `;` where applicable.

3. PROGRAM

```
bird(sparrow).  
bird(penguin).  
cannot_fly(penguin).  
can_fly(X) :- bird(X), \+ cannot_fly(X).
```

```
% Query:  
% ?- can_fly(sparrow).  
% ?- can_fly(penguin).
```

4. EXPECTED OUTPUT

```
sparrow: true. penguin: false.
```

5. CODE EXPLANATION

All birds are candidates to fly except those explicitly listed as cannot_fly.

6. WHAT TO REMEMBER

Facts + Rules + Query.

7. VIVA QUESTIONS

What is \+? Negation as failure.

Why penguin cannot fly? It is a fact in cannot_fly/1.

PROGRAM 23 — Family Tree

1. AIM

To implement Family Tree using Prolog.

2. ALGORITHM

- Define the required facts/predicates.
- Define the rule or recursive relation.
- Run the query.
- Observe the result and request alternatives with `;` where applicable.

3. PROGRAM

```
parent(raj,anu) .  
parent(anu,ram) .  
parent(raj,rita) .  
  
grandparent(X,Z):- parent(X,Y) , parent(Y,Z) .  
  
% Query:  
% ?- grandparent(raj,Z) .
```

4. EXPECTED OUTPUT

Z = ram.

5. CODE EXPLANATION

parent facts are combined by the grandparent rule.

6. WHAT TO REMEMBER

Facts + Rules + Query.

7. VIVA QUESTIONS

What is a rule? A conclusion derived from conditions.

How is grandparent derived? Parent of a parent.

PROGRAM 24 — Diet Recommendation

1. AIM

To implement Diet Recommendation using Prolog.

2. ALGORITHM

- Define the required facts/predicates.
- Define the rule or recursive relation.
- Run the query.
- Observe the result and request alternatives with `;` where applicable.

3. PROGRAM

```
disease( fever ) .
diet( fever, soup ) .
diet( fever, water ) .
diet( cold, warm_fluids ) .

recommend( D, F ) :- disease( D ) , diet( D, F ) .
```

```
% Query:
% ?- recommend( fever, F ) .
```

4. EXPECTED OUTPUT

```
F = soup ; water.
```

5. CODE EXPLANATION

Uses disease and diet facts and a rule to recommend food/fluids. This is a simple academic rule-base, not medical advice.

6. WHAT TO REMEMBER

Facts + Rules + Query.

7. VIVA QUESTIONS

What is this program? A rule-based recommendation example.

How is recommendation derived? By matching disease with diet facts.

PROGRAM 25 — Monkey-Banana Problem

1. AIM

To implement Monkey-Banana Problem using Prolog.

2. ALGORITHM

- Define the required facts/predicates.
- Define the rule or recursive relation.
- Run the query.
- Observe the result and request alternatives with `;` where applicable.

3. PROGRAM

```
state( atdoor, ground, notbanana ) .
state( atwindow, ground, notbanana ) .
state( atwindow, box, notbanana ) .

can_get( monkey, banana ) :- state( atwindow, box, notbanana ) .

% Query:
% ?- can_get( monkey, banana ) .
```


4. EXPECTED OUTPUT

true.

5. CODE EXPLANATION

This compact version represents the goal condition with facts. For a full state-space version, states and actions are expanded similarly to BFS.

6. WHAT TO REMEMBER

Represent facts/state → define rule → query.

7. VIVA QUESTIONS

What is represented? State conditions and a goal rule.

Main idea? Monkey reaches the box/banana state.

PROGRAM 26 — Fruit and Color using Backtracking

1. AIM

To implement Fruit and Color using Backtracking using Prolog.

2. ALGORITHM

- Define the required facts/predicates.
- Define the rule or recursive relation.
- Run the query.
- Observe the result and request alternatives with `;` where applicable.

3. PROGRAM

```
fruit(apple,red).  
fruit(banana,yellow).  
fruit(grape,green).  
  
find(F,C):- fruit(F,C).  
  
% Query:  
% ?- find(F,C).
```

4. EXPECTED OUTPUT

F = apple, C = red ; F = banana, C = yellow ; F = grape, C = green.

5. CODE EXPLANATION

Prolog automatically backtracks to return alternate facts on repeated ;.

6. WHAT TO REMEMBER

Pattern/list matching + Prolog backtracking.

7. VIVA QUESTIONS

What is backtracking? Prolog tries alternatives when more solutions are requested.

Predicate? fruit/2.

PROGRAM 27 — Best First Search

1. AIM

To implement Best First Search using Prolog.

2. ALGORITHM

- Define the required facts/predicates.
- Define the rule or recursive relation.
- Run the query.
- Observe the result and request alternatives with `;` where applicable.

3. PROGRAM

```
edge(a,b,2). edge(a,c,1). edge(b,d,5). edge(c,d,2).
h(a,4). h(b,3). h(c,1). h(d,0).

best(S,G):- h(S,H), write(S-H),nl, (S=G ; best_next(S,G)).
best_next(S,G):- edge(S,N,_), h(N,H), write(N-H),nl, (N=G ; best_next(N,G)).

% Query:
% ?- best(a,d).
```

4. EXPECTED OUTPUT

Shows heuristic values while moving toward d in the small example.

5. CODE EXPLANATION

Best-first search selects according to heuristic $h(n)$. The compact Prolog demonstration emphasizes the heuristic idea.

6. WHAT TO REMEMBER

Best First $\rightarrow$ smallest heuristic $h(n)$.

7. VIVA QUESTIONS

Selection value? $h(n)$.

A* difference? A* uses $g+h$; Best First uses h .

PROGRAM 28 — Medical Diagnosis

1. AIM

To implement Medical Diagnosis using Prolog.

2. ALGORITHM

- Define the required facts/predicates.
- Define the rule or recursive relation.
- Run the query.
- Observe the result and request alternatives with `;` where applicable.

3. PROGRAM

```
symptom( fever ) .  
symptom( cough ) .  
diagnosis( flu ) :- symptom( fever ) , symptom( cough ) .
```

```
% Query:  
% ?- diagnosis( flu ) .
```

4. EXPECTED OUTPUT

```
true.
```

5. CODE EXPLANATION

Symptoms are facts and diagnosis is derived by a rule. This is only a lab rule-based example, not a real diagnostic tool.

6. WHAT TO REMEMBER

Facts + Rules + Query.

7. VIVA QUESTIONS

Technique? Rule-based reasoning.

Diagnosis rule? flu if fever and cough are present.

PROGRAM 29 — Forward Chaining

1. AIM

To implement Forward Chaining using Prolog.

2. ALGORITHM

- Define the required facts/predicates.
- Define the rule or recursive relation.
- Run the query.
- Observe the result and request alternatives with `;` where applicable.

3. PROGRAM

```
fact(a).
fact(b).
rule(c):- fact(a), fact(b).
```

```
% Query:
% ?- rule(c).
```

4. EXPECTED OUTPUT

```
true.
```

5. CODE EXPLANATION

Forward chaining starts from known facts and applies rules to derive new facts toward a conclusion.

6. WHAT TO REMEMBER

Forward $\rightarrow$ Facts $\rightarrow$ Rules $\rightarrow$ Goal.

7. VIVA QUESTIONS

Direction? Facts $\rightarrow$ rules $\rightarrow$ conclusion.

Starting point? Known facts.

PROGRAM 30 — Backward Chaining

1. AIM

To implement Backward Chaining using Prolog.

2. ALGORITHM

- Define the required facts/predicates.
- Define the rule or recursive relation.
- Run the query.
- Observe the result and request alternatives with `;` where applicable.

3. PROGRAM

```
fact(a).
fact(b).
goal :- fact(a), fact(b).
```

```
% Query:  
% ?- goal.
```

4. EXPECTED OUTPUT

```
true.
```

5. CODE EXPLANATION

Backward chaining starts with a goal and checks whether the required facts can satisfy its rule.

6. WHAT TO REMEMBER

Backward → Goal → Rules → Facts.

7. VIVA QUESTIONS

Direction? Goal → required conditions → facts.

Starting point? Goal/query.

PROGRAM 31 — Word Processing — Anagram

1. AIM

To implement Word Processing — Anagram using Prolog.

2. ALGORITHM

- Define the required facts/predicates.
- Define the rule or recursive relation.
- Run the query.
- Observe the result and request alternatives with `;` where applicable.

3. PROGRAM

```
anagram(A,B):- string_chars(A,X), msort(X,Y),  
                string_chars(B,Z), msort(Z,Y).
```

```
% Query:  
% ?- anagram('listen','silent').
```

4. EXPECTED OUTPUT

```
true.
```

5. CODE EXPLANATION

Converts both words to character lists, sorts them, and compares the sorted lists.

6. WHAT TO REMEMBER

Pattern/list matching + Prolog backtracking.

7. VIVA QUESTIONS

What is anagram? Words containing the same letters in a different order.

Why msort? To sort letters for comparison.

PROGRAM 32 — Pattern Matching

1. AIM

To implement Pattern Matching using Prolog.

2. ALGORITHM

- Define the required facts/predicates.
- Define the rule or recursive relation.
- Run the query.
- Observe the result and request alternatives with `;` where applicable.

3. PROGRAM

```
match([], []).
match([H|T], [H|R]) :- match(T, R).

% Query:
% ?- match([a,b,c], [a,b,c]).
```

4. EXPECTED OUTPUT

```
true.
```

5. CODE EXPLANATION

Pattern matching succeeds when corresponding list elements are equal and both lists end together.

6. WHAT TO REMEMBER

Pattern/list matching + Prolog backtracking.

7. VIVA QUESTIONS

What is [H|T]? Head H and tail T of a list.

When does it fail? When corresponding elements differ.

PROGRAM 33 — Number of Vowels

1. AIM

To implement Number of Vowels using Prolog.

2. ALGORITHM

- Define the required facts/predicates.
- Define the rule or recursive relation.
- Run the query.
- Observe the result and request alternatives with `;` where applicable.

3. PROGRAM

```
vowel(a). vowel(e). vowel(i). vowel(o). vowel(u).  
  
count([],0).  
count([H|T],N):- vowel(H), count(T,N1), N is N1+1.  
count([H|T],N):- \+ vowel(H), count(T,N).  
  
% Query:  
% ?- count([a,b,e,c,i],N).
```

4. EXPECTED OUTPUT

N = 3.

5. CODE EXPLANATION

Recursively checks each character. Vowels increase the count; other characters leave it unchanged.

6. WHAT TO REMEMBER

Prolog recursion → Base case + Recursive case.

7. VIVA QUESTIONS

Base case? count([],0).

Vowels? a, e, i, o, u.

FINAL 1-PAGE MEMORY SHEET

- BFS → Queue → popleft() → visited.
- DFS → Recursion / Stack → visited.
- 8-Queen / CSP → Backtracking → Choose → Check → Recurse → Undo.
- Water Jug → State (x,y) + Fill/Empty/Transfer.
- Missionaries → State (M,C,B) + Safe check.
- 8-Puzzle → State/Board + blank moves + BFS.
- TSP → Permutations + total cost + minimum.
- Crypt-Arithmetic → Unique digits + permutations + equation.
- A* → $f(n) = g(n) + h(n)$.
- Best First → $h(n)$.
- Minimax → MAX=max, MIN=min.
- Alpha-Beta → $\alpha \geq \beta$ → prune.
- Decision Tree → fit() → predict().
- MLP → fit() → predict().
- Prolog database → Facts + Rules + Query.
- Prolog recursion → Base case + Recursive case.
- Forward chaining → Facts → Goal.
- Backward chaining → Goal → Facts.
- Prolog backtracking → ask for next solution with ;.